

1. Project Description

This project involves the design and simulation of a digital stopwatch using digital logic components. The stopwatch is capable of counting time in minutes and seconds up to 59 minutes 59 seconds and displaying the output using 4 7-segment displays. The system is implemented using BCD counters -> BCD-to-7-segment decoders, logic gates for flow control, and a clock generator in the Multisim environment.

The main objective of this project is to create a functional stopwatch system. The stopwatch works based on clock pulses, where each pulse represents one second. These pulses are counted and displayed in 4 7-segment displays, with separate counters for seconds and minutes. Control inputs such as start/stop and reset allow the user to use the stopwatch.

This project is highly relevant to digital systems as it demonstrates the practical use of counters and display interface. Time keeping circuits are widely used in real world applications such as digital clocks, timers, embedded systems, and industrial control systems. Through this project, a deeper understanding of cascading counters and signal synchronization is achieved.

Overall, the stopwatch serves as a detailed example of integrating multiple digital components into a single and functional system.

2. Project Flow

Stopwatch Overview

The digital stopwatch operates based on a continuous clock signal that drives a series of connected BCD counters. The system responds to user inputs (Start/Stop and Reset) and updates the displayed time accordingly. The counting process follows a sequential flow where seconds are incremented first, followed by minutes.

Flow Description

The stopwatch starts by resetting all counters to 00:00. It stays idle until the Start/Stop signal is enabled, allowing a 1 Hz clock to drive the circuit. Each pulse increases the seconds unit counter (0–9), which triggers the seconds tens counter (0–5). When time reaches 59 seconds, both seconds counters reset and a carry signal increments the minutes counter. Minutes are counted similarly, with units (0–9) and tens (0–5). The BCD outputs are decoded to 7-segment displays showing (MM:SS). When Stop is activated, counting pauses while the displayed value is held. The system repeats this cycle while running.

Process Summary

- Inputs:

- Clock signal (1 Hz)
- Start/Stop control
- Reset button

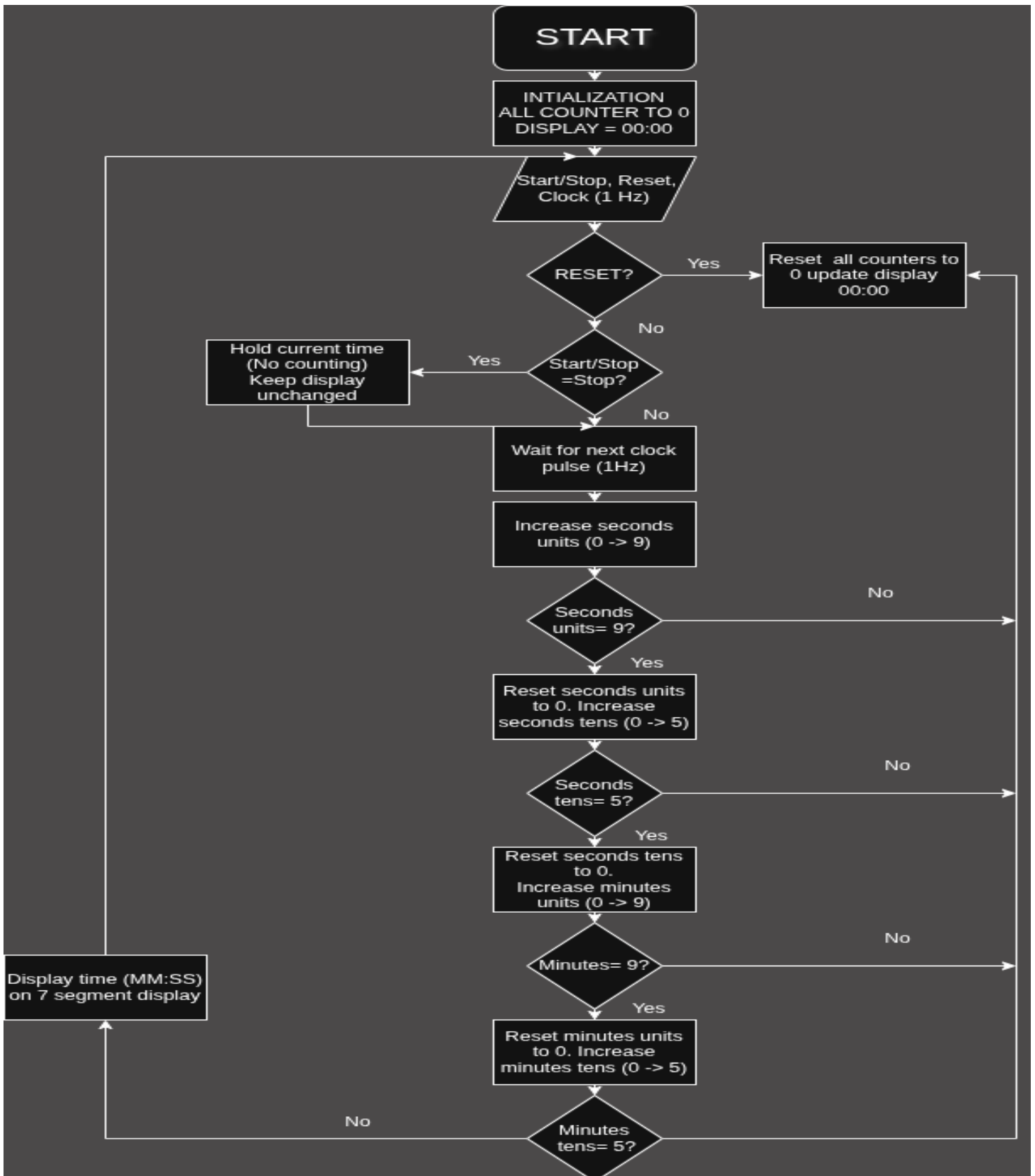
- Processing:

- Counting using BCD counters
- Carry propagation between counters
- Reset logic for mod-10 and mod-6 operation

- Outputs:

- Time displayed on 7-segment displays (MM:SS format)

Flow chart



3. Component Functionality

BCD Counter (74LS90)

The 74LS90 is a decade (mod-10) counter used to count clock pulses in BCD form. In this project, many 74LS90 counters are used to represent the units and tens digits of seconds and minutes.

- **Inputs:** Clock signal, reset inputs, stop/pause inputs
- **Outputs:** Four-bit BCD output
- **Function:** Counts incoming clock pulses and produces a binary output corresponding to decimal digits.
- **Usage in circuit:** Counters (0–9) Tens of seconds and minutes are set to (0–5) using and gates.
- **Reason for selection:** Simple, reliable, and widely used for decimal counting in digital systems

BCD to 7 Segment Decoder (7447)

The 7447 decoder converts BCD inputs into signals required to drive a 7 segment display.

- **Inputs:** 4-bit BCD input
- **Outputs:** Seven control signals (a–g) for display segments
- **Function:** Translates binary values into readable decimal digits
- **Usage in circuit:** Each counter output is connected to a decoder to display the corresponding digit
- **Reason for selection:** Specifically designed for common 7 segment displays and simplifies display interfacing

7-Segment Display

The 7-segment display is used to visually represent numerical values.

- **Inputs:** Signals from the decoder (segments a–g)
- **Function:** Displays decimal digits (0–9)
- **Usage in circuit:** Four displays are used to show minutes and seconds in (MM:SS) format
- **Reason for selection:** Simple and effective method for numerical output in digital systems

Clock Generator

The clock generator produces a periodic signal that drives the counting process.

- **Output:** Square wave signal (1 Hz)
- **Function:** Provides timing pulses for the counters to change
- **Usage in circuit:** Each pulse should increase the seconds counter by one unless paused
- **Reason for selection:** Ensures consistent and accurate timing for the stopwatch operation

Interactive Digital Constant (Start/Stop and Reset)

Interactive Digital Constant are used to control the operation of the stopwatch.

- **Start/Stop:** Enables or disables counting by controlling the input signal
- **Reset:** Clears all counters and returns the display to 00:00
- **Function:** Allows user interaction with the system
- **Usage in circuit:** Connected to logic that controls counting and resetting behavior
- **Reason for selection:** Provides simple control of the stopwatch

Logic Gates (AND, OR)

Logic gates are used to implement control and reset conditions within the circuit.

- **AND Gates:** Used for enabling signals and detecting specific count conditions
- **OR Gates:** Used to combine multiple control signals
- **Function:** Condition enforcement in the circuit
- **Usage in circuit:** Generating reset signals for tens counters Etc
- **Reason for selection:** Simplest way to detect and control conditions

Power Supply (VCC)

The power supply provides the required operating voltage for all components.

- **Voltage:** +5V
- **Function:** Powers all ICs and ensures functionality
- **Reason for selection:** Standard power supply

4. Components List

1. 74LS90N counter IC x4
2. 7447N 4bit BCD to 7 segment Decoder x4
3. Seven segment display blue x4
4. VCC 5 volt x9
5. AND2 gates x5
6. AND4 gate

7. OR2 gate x2
8. Interactive Digital Constant x2
9. Digital clock

5. Circuit Analysis

Circuit Behavior

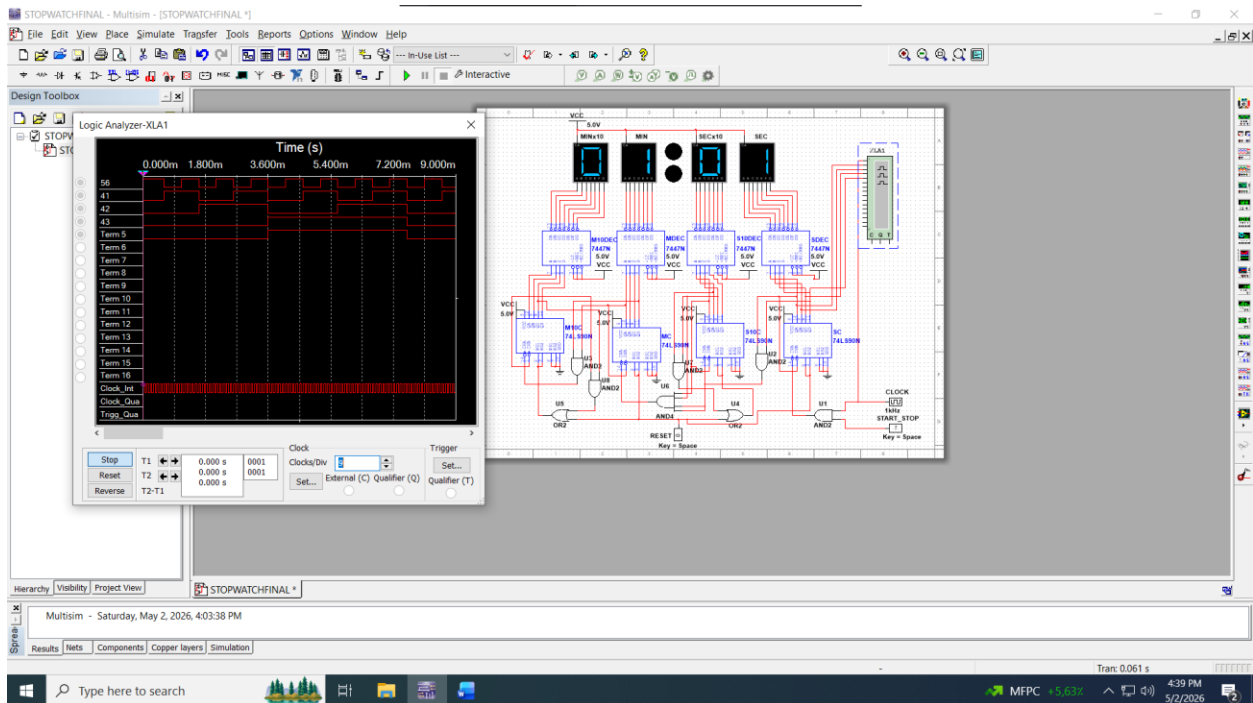
The stopwatch operates using a 1 Hz clock signal that drives the first BCD counter. Each clock pulse increases the seconds unit digit. When the count reaches 9, a carry signal increments the seconds tens counter. This process continues until 59 seconds, after which the seconds reset to 00 and a carry signal increments the minutes counters.

The Start/Stop control enables or disables the clock signal, allowing the stopwatch to pause and resume counting. The Reset input forces all counters to zero, displaying 00:00.

Simulation Results

Simulation was performed using Multisim. A logic analyzer was used to observe signal behavior.

- The clock signal produced a stable square wave at 1 Hz.
- The output of the first 74LS90 showed binary counting behavior.
- Carry signals were observed when transitioning from 9 to 0.
- The 7 segment displays correctly showed time progression from 00:00 upward.



(Clock was set to 1KHz to speed up testing in a real life application it should be set to 1Hz.)

Expected vs Actual Results

Feature	Expected	Actual
Clock	1 Hz pulse	Correct
Counting	Increment every second	Correct
Reset	Return to 00:00	Correct
Carry	Trigger next counter	Correct

Inconsistencies

Minor delays were observed during transitions due to propagation delay in the 74LS90 counters. These delays did not significantly affect overall functionality.

Performance

The stopwatch operates accurately with stable counting and correct display output. The modular design allows easy expansion.

Strengths and Limitations

Strengths:

- Accurate counting
- Clear display output
- Simple modular design

Limitations:

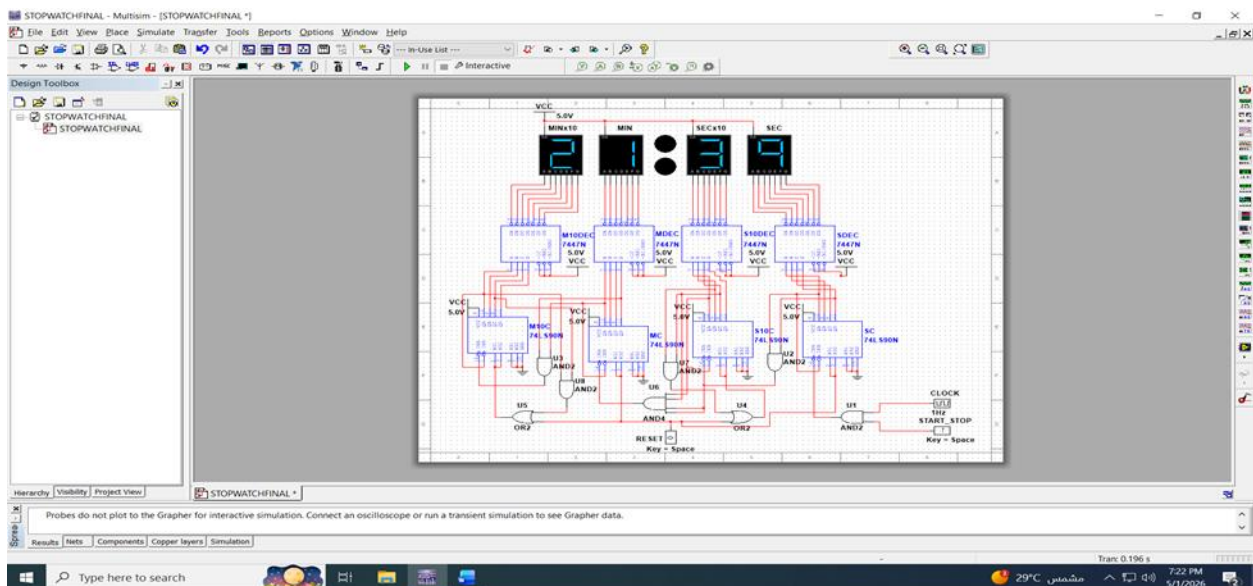
- Dependent on clock accuracy
- No buttons for control inputs instead relies on interactive inputs to start/pause/reset
- Limited to (MM:SS) format

6. Project Documentation

Simulation Diagram

The complete blue print of the digital stopwatch was designed and simulated using Multisim. The circuit consists of four BCD counters (74LS90), four BCD-to-7-segment decoders (7447), four 7-segment displays, logic gates, a clock source, and control inputs for start/stop and reset.

The simulation clearly shows the connection between the counters, decoders, and displays, making a complete time counting system that displays minutes and seconds in (MM:SS) format.



Connections

The circuit is organized in a sequential structure to implement carry counting in counters

- The clock signal is connected to the seconds unit counter, which increases with every clock pulse.
- The seconds unit counter (0–9) generates a carry signal that increases the seconds tens counter.
- The seconds tens counter (0–5) is set using AND gate to reset it after reaching 5.
- When the count reaches 59 seconds, a reset signal is generated that clears both seconds counters and sends a carry to the minutes unit counter using AND gate.
- The minutes unit counter (0–9) increases accordingly, and it sends a carry to the minutes tens counter.

The outputs (Q0–Q3) of each counter are connected to the corresponding inputs of the 7447 decoders, which then drives the 7-segment displays.

The reset input is connected to all counters, allowing the system to return to 00:00 instantly. The start/stop control is implemented using an interactive digital input to enable or pause the counting process. On the tens counter(seconds/minutes) there is an OR gate one pin is connected to the AND gate for the 5 reset condition and the other pin is connected to the interactive constant reset input to allow it to reset when either condition is met.

Pin Configuration

Each 74LS90 counter provides four outputs (Q0–Q3), representing a BCD value. These outputs are directly connected to the A, B, C, and D inputs of the 7447 decoders.

Reset pins on the counters are used to implement specific counting limits normal counting for seconds and additional AND gates are used for tens digits.

The 7447 decoder outputs are connected to the segments of the 7-segment displays, which are set to send the outputs (0-7) to (a-g).

Design Logic

The stopwatch is based on sequential counting using BCD counters and carry logic.

- The seconds/minutes unit counter operates as a counter (0–9).
- The seconds/minutes tens counter is set as a (0–5) to ensure valid time representation.
- When the count reaches 59 seconds, the system resets the seconds counters to 00 and increments the minutes counter.

This is achieved by detecting the binary condition corresponding to 59 and generating a reset signal using AND gates.

Testing and Debugging

During the development of the circuit, several challenges were met and resolved:

1. Initially, the start/stop functionality using logic gates did not behave properly, so it was replaced with an interactive digital input, which provided stable and functional control.
2. The circuit required multiple redesigns due to wiring complexity, and 5 iterations were performed to improve clarity in presentation and organization.
3. The implementation of tens counters required redesign with OR gates to ensure correct behavior at the transition from 59 to 00 while maintaining a reset input.
4. While using the logic analyzer, the signals initially appeared inactive due to Multisim slow down the simulation time to real time this was solved by increasing the clock Hz and increasing the logic analyzer time window.

Through testing and debugging, the circuit was verified to work correctly, with accurate counting, proper reset behavior, and stable display output.

7. VHDL

Stopwatch source code:

```
library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity stopwatch is

    Port (

        clock    : in  STD_LOGIC;

        reset    : in  STD_LOGIC;

        enable   : in  STD_LOGIC;

        sec_u    : out INTEGER range 0 to 9;

        sec_t    : out INTEGER range 0 to 5;

        min_u    : out INTEGER range 0 to 9;

        min_t    : out INTEGER range 0 to 5

    );

end stopwatch;
```

architecture Behavioral of stopwatch is

signal s_u, s_t, m_u, m_t : INTEGER := 0;

begin

process(clock, reset)

begin

if reset = '1' then

s_u <= 0;

s_t <= 0;

m_u <= 0;

m_t <= 0;

elsif rising_edge(clock) then

if enable = '1' then

if s_u = 9 then

s_u <= 0;

if s_t = 5 then

s_t <= 0;

if m_u = 9 then

m_u <= 0;

if m_t = 5 then

m_t <= 0;

else

```

        m_t <= m_t + 1;
    end if;

    else

        m_u <= m_u + 1;
    end if;

    else

        s_t <= s_t + 1;
    end if;

    else

        s_u <= s_u + 1;
    end if;

    end if;

end if;

end process;

sec_u <= s_u;
sec_t <= s_t;
min_u <= m_u;
min_t <= m_t;

end Behavioral;

```

Stopwatch simulation testing code:

```

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

```

entity stopwatch_tb is

end stopwatch_tb;

architecture Behavioral of stopwatch_tb is

-- Component declaration

component stopwatch

Port (

clock : in STD_LOGIC;

reset : in STD_LOGIC;

enable : in STD_LOGIC;

sec_u : out INTEGER range 0 to 9;

sec_t : out INTEGER range 0 to 5;

min_u : out INTEGER range 0 to 9;

min_t : out INTEGER range 0 to 5

);

end component;

-- Signals

signal clock : STD_LOGIC := '0';

signal reset : STD_LOGIC := '1';

signal enable : STD_LOGIC := '0';

signal sec_u : INTEGER;

signal sec_t : INTEGER;

signal min_u : INTEGER;

signal min_t : INTEGER;

begin

```
-- Instantiate the stopwatch
```

```
uut: stopwatch
```

```
port map (
```

```
    clock => clock,
```

```
    reset => reset,
```

```
    enable => enable,
```

```
    sec_u => sec_u,
```

```
    sec_t => sec_t,
```

```
    min_u => min_u,
```

```
    min_t => min_t
```

```
);
```

```
-- Clock generation (fast for simulation)
```

```
clk_process : process
```

```
begin
```

```
    clock <= '0';
```

```
    wait for 10 ns;
```

```
    clock <= '1';
```

```
    wait for 10 ns;
```

```
end process;
```

```
-- Stimulus process
```

```
stim_proc: process
```

```
begin
```

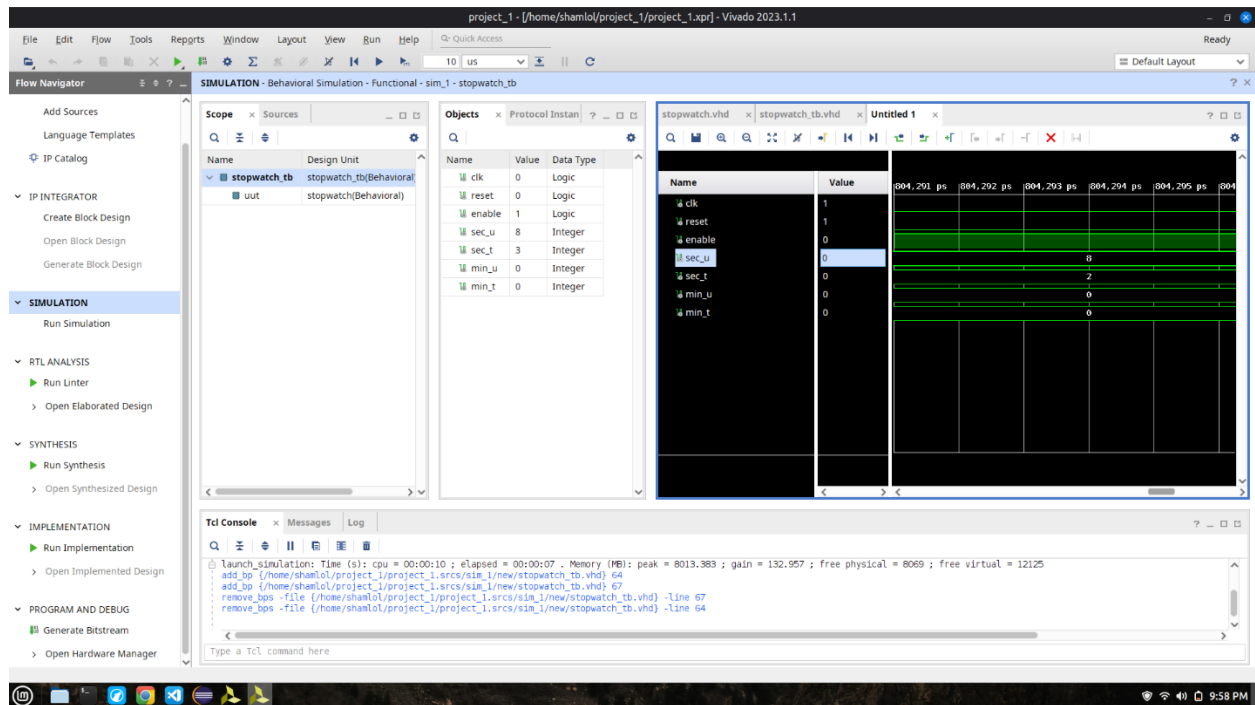
```
    -- Initial reset
```

```
    wait for 50 ns;
```

```
    reset <= '0';
```

```
    enable <= '1';
```

```
-- Run for some time  
wait for 500 ns;  
  
-- Pause  
enable <= '0';  
wait for 200 ns;  
  
-- Resume  
enable <= '1';  
wait for 500 ns;  
  
-- Reset again  
reset <= '1';  
wait for 50 ns;  
reset <= '0';  
  
wait;  
end process;  
  
end Behavioral;
```



Results:

The simulation waveforms (as seen in the Vivado screenshot) confirm that the counters increment sequentially and roll over at the correct timings. The sec_u, sec_t, and min signals update only when the previous stage completes its cycle, proving the cascading logic is functional.